

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset $\text{\LaTeX}$ solutions.

1.a

Given that $m, n \geq 0$, the goal location (m, n) lies in the northeast quadrant relative to the start location $(0, 0)$. Therefore, any shortest path must eventually increase x exactly m times (via East moves) and increase y exactly n times (via North moves). Moving West or South would only make the path longer because it increases the number of steps needed to reach (m, n) .

The cost of any move is

$$\text{Cost}((x, y), a) = 1 + \max(x, 0),$$

which depends only on the current x -coordinate, not on the direction of movement. For this reason, the order of moves matters: moving North while $x = 0$ costs only 1, whereas moving North after increasing x would cost more. Thus, the optimal strategy is to perform all n North moves first while $x = 0$, and then perform the m East moves, hence, being a unique path.

The m East moves begin at $x = 0, 1, 2, \dots, m - 1$, so their total cost is

$$(1 + 0) + (1 + 1) + \dots + (1 + (m - 1)) = m + \sum_{k=0}^{m-1} k = m + \frac{m(m-1)}{2} = \frac{2m + m(m-1)}{2} = \frac{m(m+1)}{2}.$$

The n North moves (all done at $x = 0$) each cost 1, giving total cost n .

Therefore, the minimum cost of reaching (m, n) is

$$\text{MinCost}(m, n) = n + \frac{m(m+1)}{2}.$$

1.b

(i) *False.*

(ii) *False*

(iii) *True*

1.c

(i) *True*

(ii) *False*

(iii) *False*

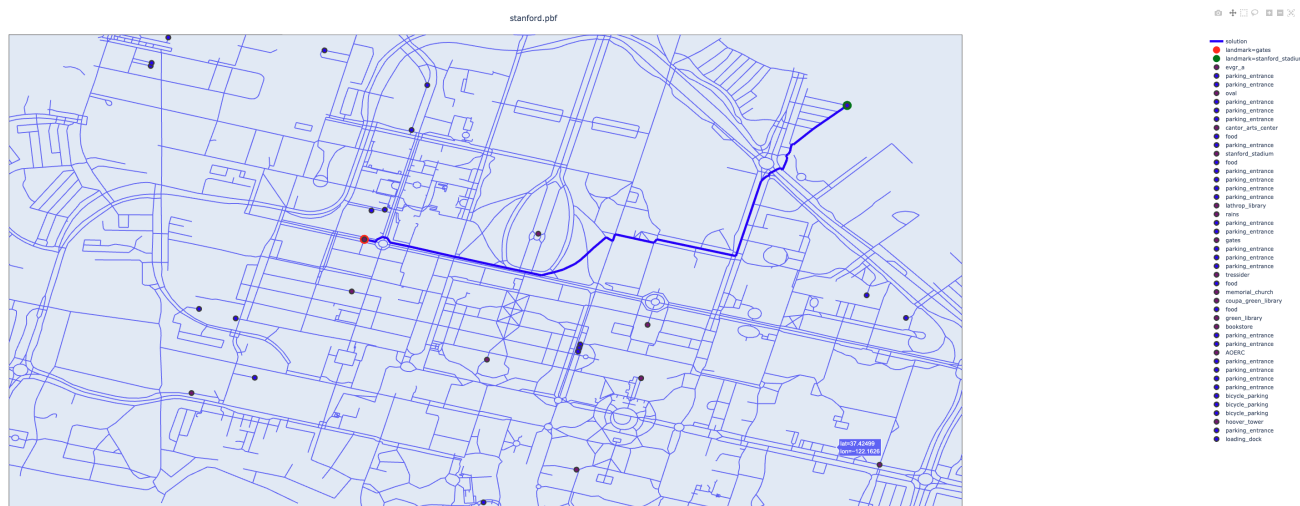
2.b

Figure 1: Route from “gates” to “cantor arts center”



As it can be seen in Figure 3, the route is traced properly, looking straightforward and without loops or undesired detours. The algorithm identifies the shortest connection between the two points based purely on distance.

Figure 2: Route from “gates” to “stanford stadium”



Even if this route in Figure 2 appears appropriate in terms of distance, as in the previous case, there are several additional criteria that could be taken into account. The algorithm optimizes solely for geographical distance, treating all edges equally regardless of walkway width, comfort or foot-traffic suitability. Thus, a visually reasonable path on the graph may still differ from what a real pedestrian would choose in practice. It might not be the case with this particular route, due to my lack of familiarity with the site, but it could be something to take into account for the algorithm.

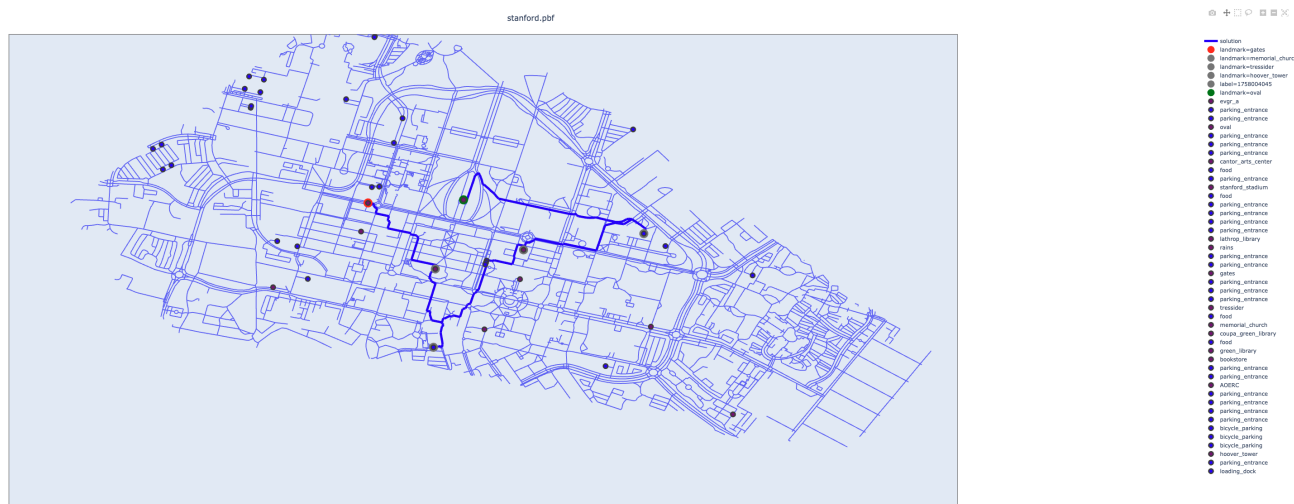
3.b

In this problem a state keeps track of two things: the current location and which waypoint tags (out of the k given) have already been covered. For the waypoint tags, any combination may occur during the search, so there are 2^k possible subsets. Since each of the n locations can be paired with any of these subsets, the maximum number of states that UCS might explore is

$$n \cdot 2^k.$$

3.c

Figure 3: Route from “gates” to “Oval” through 4 additional waypoints



For my 3(c) example, I used the following waypoint tags:

```
landmark=hoover_tower,  
landmark=tressider,  
label=1758004045,  
landmark=memorial_church.
```

The visualized route works well and reaches all the waypoints in a reasonable distance. Still, the model only focuses on distance and does not consider how comfortable or practical each segment is for a pedestrian, so the resulting path may not always reflect what someone would naturally choose.

Note: I chose the label 1758004045 to deviate a bit further the route and test the algorithm a bit better.

4.d

If the waypoints are placed along the diagonal of the grid, where $x = y$ for each waypoint, then any shortest path from " $0, 0$ " to " $(n - 1, n - 1)$ " naturally goes through all of them. In this situation, the waypoints do not force any detours, so the shortest path with waypoints has the same cost as the relaxed shortest path. Because the NoWaypointsHeuristic equals the true remaining cost in this case, A explores states in the same order as UCS, and both algorithms behave the same in terms of running time.